LUXUS CAPITAL

SUB-BRAND E-COMMERCE PLATFORM

Technical Scope of Work & Architecture Specification

Composable Headless Commerce Platform
for High-End Firearms Retail

Prepared: April 2026

Version: 1.1

Classification: Confidential

Table of Contents

This document provides a complete technical specification for the Luxus Capital sub-brand e-commerce platform, covering architecture, technology stack, feature specifications, development phases, cost estimates, and operational requirements.

1. Executive Summary

This document specifies the technical architecture, software stack, and development scope for a new e-commerce platform operating as a sub-brand under Luxus Capital, LLC. The platform will sell high-end production firearms from manufacturers including Nighthawk Custom, Cabot Guns, and similar premium brands. It will operate as a separate domain from luxuscap.com (the existing curated gallery), with strategic cross-site linking between the two properties.

The recommended architecture is a composable headless commerce stack built on open-source foundations (Medusa.js, Next.js, Payload CMS), paired with best-in-class specialized services for payments, email marketing, video, 3D visualization, and AI integration. This approach provides maximum design control, zero platform-level firearms restrictions, full data ownership, and a future-proof foundation for agentic commerce protocols (MCP, UCP, ACP) as they mature.

The platform is designed to be industry-leading in the firearms e-commerce space, incorporating capabilities that no competitor currently offers: interactive 3D product visualization with AR, cinematic video product tours, an MCP server for AI agent discoverability, rich structured data for search engines and LLMs, a mobile-first Progressive Web App experience with push notifications and offline browsing, and a premium editorial content experience that matches the caliber of the products being sold.

2. Project Overview & Objectives

2.1 Business Context

Luxus Capital, LLC is an FFL holder that currently operates luxuscap.com as a curated gallery of historically significant and master-grade collectible firearms. The existing site is SEO-optimized as a gallery and reference resource, not a transactional store. The new sub-brand platform will serve as the commerce arm, listing high-end production firearms for direct sale.

2.2 Key Constraints

Firearms Industry Restrictions: Major e-commerce platforms (Shopify, Squarespace, Wix) and payment processors (Stripe, PayPal, Square) prohibit firearms sales. The platform must use firearms-friendly infrastructure at every layer.

Single FFL, Own Inventory: Luxus Capital is the seller and FFL holder. No distributor drop-shipping or marketplace functionality is required. Inventory is managed directly by the Luxus team.

Premium Brand Positioning: The sub-brand must match the design quality and editorial tone of luxuscap.com. Template-based or generic firearms platform aesthetics are not acceptable.

AI-Forward Architecture: The platform must be built with agentic commerce in mind: API-first architecture, MCP server integration, structured data for LLM consumption, and llms.txt compliance.

2.3 Core Objectives

- Sell high-end production firearms (Nighthawk, Cabot, and similar) through a premium, brand-controlled online storefront
- Accept credit card payments via firearms-friendly processors, plus wire transfer and check as primary payment methods for high-ticket items
- Provide industry-leading product presentation: photography, video, 3D/AR visualization
- Build a content engine (blog, news, manufacturer profiles) that drives organic traffic and cross-links with luxuscap.com
- Implement email marketing automation with segmented campaigns, newsletter, and transactional notifications
- Expose product catalog to AI agents via MCP server and structured data for future agentic commerce
- Maintain full ATF/FFL compliance with FastBound integration for A&D book and FFL transfer workflows

3. Technology Stack Summary

The following table provides a high-level overview of every software component, service, and technology in the platform. Each is covered in detail in subsequent sections.

Layer	Technology	License/Cost	Purpose
Commerce Engine	Medusa.js 2.x	MIT (Free)	Product catalog, cart, checkout, orders, inventory, tax
Front-End Framework	Next.js 15+	MIT (Free)	Server-rendered React storefront, SEO, performance
Content Management	Payload CMS 3.x	MIT (Free)	Blog, articles, manufacturer pages, editorial content
Payment Processing	EPIC / TacticalPay / PaymentCloud	$19.99+/mo + fees	Credit card processing for firearms
Wire/Check Payments	Custom Medusa Plugin	Included in build	Manual payment methods for high-ticket sales
FFL Compliance	FastBound	~$39-99/mo	Digital A&D book, ATF compliance, FFL verification
Email Marketing	Klaviyo	$20+/mo	Newsletter, automation flows, segmentation, campaigns
Transactional Email	Resend + React Email	$20+/mo	Order confirmations, inquiry notifications, branded emails
Video Platform	Mux	Usage-based (~$20+/mo)	Adaptive streaming, branded player, no YouTube branding
3D Visualization	Google model-viewer	Free (Open Source)	Interactive 3D product viewer, AR on mobile
3D Rendering	React Three Fiber	MIT (Free)	Advanced interactive 3D experiences, hotspots
3D Asset Creation	Polycam / RealityScan	$0-50/mo	Photogrammetry pipeline for 3D model creation

Search	Meilisearch	MIT (Free)	Full-text product search, faceted filtering
SEO	next-seo + Custom	Included in build	Meta tags, OpenGraph, canonical URLs, JSON-LD
Schema Markup	Custom JSON-LD	Included in build	Product, Organization, Article schema for Google/AI
LLM Integration	Custom llms.txt generator	Included in build	Auto-generated from product database
MCP Server	Custom (TypeScript SDK)	Included in build	AI agent catalog access, agentic commerce foundation
Internal Linking	Custom Payload Plugin	Included in build	Rule-based keyword-to-URL auto-linking in content
Analytics	Plausible or Fathom	$9-14/mo	Privacy-respecting, cookie-free analytics
Hosting (Front-End)	Vercel	$20-150/mo	Edge deployment, SSR, automatic scaling
Hosting (Backend)	Railway or Render	$20-100/mo	Medusa.js, Payload CMS, MCP server
Database	PostgreSQL (Neon or Supabase)	$0-25/mo	Primary data store for Medusa and Payload
File Storage	AWS S3 / Cloudflare R2	$5-20/mo	Product images, videos, 3D assets, documents
CDN	Cloudflare	Free-$20/mo	Asset delivery, DDoS protection, edge caching
Domain & DNS	Cloudflare Registrar	~$10/yr	Domain registration and DNS management
SSL/TLS	Cloudflare / Let's Encrypt	Free	HTTPS encryption
Monitoring	Sentry	Free-$26/mo	Error tracking, performance monitoring
Version Control	GitHub	Free-$4/user/mo	Code repository, CI/CD pipelines

PWA Framework	next-pwa / Serwist	MIT (Free)	Service worker, offline caching, install prompt
Push Notifications	Web Push API + custom backend	Included in build	Back-in-stock alerts, order updates, new arrivals
CSS Framework	Tailwind CSS	MIT (Free)	Utility-first responsive design, mobile breakpoints
Performance CI	Lighthouse CI	Free (Open Source)	Automated Core Web Vitals testing on every deploy

4. Core Architecture: Commerce Engine (Medusa.js)

Medusa.js is an open-source headless commerce engine built on Node.js and TypeScript. It provides the transactional backbone of the platform: product catalog management, inventory tracking, cart and checkout flows, order management, customer accounts, tax calculation, and shipping logic. All functionality is exposed through a RESTful API that the Next.js front-end consumes.

4.1 Why Medusa.js

No platform-level firearms restrictions. Medusa is open-source software you self-host. There is no acceptable-use policy, no account to be suspended, and no vendor who can de-platform you. You own the code and the data.

Modular, plugin-based architecture. Medusa uses a module system where you add capabilities as needed: payment plugins, notification plugins, fulfillment plugins. The firearms-specific logic (FFL transfer workflow, state restriction engine, FastBound integration) is built as custom Medusa modules that slot cleanly into the existing architecture.

API-first by design. Every piece of data and every action in Medusa is available via API. This is critical for the MCP server (which queries the same APIs to serve AI agents), for the Klaviyo integration (which listens to order events), and for future agentic commerce protocols.

TypeScript throughout. The commerce engine, the front-end (Next.js), the CMS (Payload), and the MCP server all share the same language. This means one developer can work across the entire stack, shared type definitions reduce bugs, and AI-assisted development tools work more effectively with a consistent codebase.

4.2 Key Medusa Features Used

Product Catalog: Rich product records with title, description, images, variants (caliber, finish, barrel length), metadata (manufacturer, provenance, serial), categories, tags, and collections. Supports custom attributes for firearms-specific data.

Inventory Management: Real-time stock tracking with quantity, stock status (in stock, sold, reserved, on hold), and price management. Admin dashboard for easy updates to price, quantity, and availability.

Cart & Checkout: Full cart management with line items, discounts, tax calculation, and multi-step checkout. Custom checkout steps for FFL dealer selection and state compliance verification.

Order Management: Order creation, fulfillment tracking, status updates, refund processing. Integrates with transactional email for customer notifications at each stage.

Customer Accounts: Registration, login, order history, saved payment methods (via processor), and wishlist/favorites. Customer data syncs to Klaviyo for marketing segmentation.

Admin Dashboard: Built-in admin UI for the Luxus team to manage products, orders, customers, and settings without developer involvement.

4.3 Custom Medusa Modules Required

Firearms Payment Plugin: Custom module integrating the chosen firearms-friendly payment processor's API (EPIC, TacticalPay, or PaymentCloud). Handles credit card authorization, capture, and refund flows.

Wire/Check Payment Plugin: Manual payment method that creates orders in "pending payment" state. Includes admin actions to mark payment received and release for fulfillment. 72-hour item reservation logic.

FFL Transfer Module: Custom checkout step where buyers select their receiving FFL dealer. Integrates with FFL lookup services (FFLScope or FFL API) to verify active licenses. Stores selected FFL on the order record.

State Restriction Engine: Rule-based system that blocks or warns on product/state combinations. Configurable per product or category. Enforced at cart/checkout level. Examples: no specific magazines to certain states, no NFA items without proper documentation.

FastBound Integration Module: Syncs acquisitions and dispositions to FastBound's digital A&D book via their REST API. Automatically logs when inventory is received and when items are shipped to FFL dealers.

5. Front-End Framework (Next.js)

Next.js is a React-based framework that provides server-side rendering (SSR), static site generation (SSG), and edge deployment. It serves as the customer-facing storefront that consumes data from both Medusa.js (commerce) and Payload CMS (content).

5.1 Key Capabilities

Server-Side Rendering: Every product page, category page, and article renders complete HTML on the server before sending to the browser. This ensures fast first-paint for customers and full content visibility for search engine and AI crawlers. JavaScript-heavy SPAs are invisible to many AI bots; SSR solves this.

Edge Deployment (Vercel): Pages are served from the nearest edge node globally. Target Time to First Byte (TTFB) under 200ms. AI crawlers are impatient and move on from slow responses; edge deployment prevents this.

Image Optimization: Next.js has built-in image optimization that automatically serves WebP/AVIF formats, lazy-loads below-fold images, and generates responsive srcsets. Critical for product photography performance.

App Router with React Server Components: Next.js 15+ uses the App Router architecture where data fetching happens on the server, reducing client-side JavaScript and improving performance.

5.2 Page Types

Page Type	Description
Homepage	Featured products, brand highlights, latest blog posts, newsletter signup
Product Listing (PLP)	Filterable grid by brand, caliber, category, price. Faceted search via Meilisearch
Product Detail (PDP)	Photography gallery, video embed (Mux), 3D viewer (model-viewer), specs table, inquiry form, schema markup, related products
Brand/Manufacturer Page	Editorial profile, history, philosophy, linked products. Content from Payload CMS
Blog/Article Page	Rich content with internal auto-linking, related products, author, publish date
Cart Page	Line items, quantities, running total, proceed to checkout
Checkout Flow	Multi-step: shipping info, FFL selection (for firearms), payment method, order review, confirmation
Account Dashboard	Order history, tracking, saved details, wishlist
Contact/Inquiry Page	General contact form, linked to specific product if accessed from PDP
Legal Pages	Terms, privacy policy, FFL information, shipping policy, return policy

6. Content Management System (Payload CMS)

Payload CMS is an open-source, self-hosted headless CMS built on Node.js and TypeScript. It provides the editorial backend for all non-commerce content: blog posts, articles, manufacturer profiles, resource pages, and marketing landing pages.

6.1 Content Types (Collections)

Blog Posts / Articles: Title, slug, rich text body, featured image, excerpt, author, publish date, categories, tags, SEO fields (meta title, meta description, OpenGraph image, canonical URL, schema type selector).

Manufacturer Profiles: Brand name, logo, hero image, rich text history/philosophy, location, founding year, gallery images, video embed, linked products (relationship to Medusa products), SEO fields.

Resource Pages: Flexible page builder with content blocks: text, image, video, product grid, FAQ accordion, comparison table. Used for buying guides, collector education, and reference content.

Newsletter Campaigns: Content drafts that feed into Klaviyo campaigns. Rich text body, product highlights, CTA buttons.

Internal Linking Rules: Keyword-to-URL mapping table. The internal linking engine scans blog content at publish time and auto-inserts links based on these rules (e.g., "Nighthawk Custom" always links to the manufacturer profile page; "1911" links to the 1911 category page).

6.2 SEO Fields (Per Content Type)

Every content type in Payload includes a reusable SEO field group with the following fields: Meta Title (with character count and preview), Meta Description (with character count and preview), OpenGraph Title, OpenGraph Description, OpenGraph Image, Canonical URL override, Schema Type selector (Article, Product, FAQPage, CollectionPage, etc.), noindex/nofollow toggles, and a focus keyword field for on-page optimization guidance.

7. Payment Processing

Payment processing for firearms is a specialized requirement. Standard processors (Stripe, PayPal, Square) prohibit firearms transactions. The platform must integrate with processors that explicitly serve the 2A industry.

7.1 Recommended Processors (Evaluate All Three)

Processor	Key Features	Integration	Notes
EPIC Merchant Systems	Zero-cost processing (customer pays fee), Clover POS integration, FastBound integration, no high-risk classification	REST API, gateway integration with WooCommerce/BigCommerce/custom	Veteran-owned, strong FFL focus
TacticalPay	ACH, debit, and credit processing, plug-and-play gateways, virtual terminal for phone orders	Gateway API compatible with major carts, custom API available	Leader in firearms payment processing
PaymentCloud	High-risk merchant accounts, supports all major card brands, chargeback protection	Multiple gateway options, API integration available	Broader high-risk expertise beyond firearms

7.2 Wire Transfer & Check Payments

For high-ticket firearms ($3,000-$50,000+), wire transfer is often the preferred payment method for both buyer and seller. The platform implements wire/check as a first-class payment experience, not a fallback.

Checkout Flow: Customer selects "Wire Transfer" or "Check" at payment step. Order is created in "Pending Payment" status. Item is reserved (held from other buyers) for a configurable period (default: 72 hours for wire, 10 business days for check). Customer receives branded confirmation email with wire instructions or mailing address.

Admin Flow: Team monitors incoming payments. Admin marks order as "Payment Received" in Medusa dashboard. Order moves to fulfillment queue. If payment is not received within the reservation window, order is auto-cancelled and inventory is released.

8. Compliance & FFL Integration (FastBound)

FastBound is the industry-standard digital A&D (Acquisition and Disposition) bound book for FFL dealers. Integration with the e-commerce platform ensures that every transaction is automatically logged for ATF compliance.

FastBound REST API: The custom Medusa module communicates with FastBound's API to log acquisitions (when inventory is added to the platform) and dispositions (when items are sold and shipped to receiving FFLs). Serial numbers, manufacturer, model, caliber, and transaction details are synced automatically.

FFL Dealer Lookup: At checkout, customers who are purchasing a firearm (not accessories) select their receiving FFL dealer. The platform integrates with FFLScope or the FFL EZ Check API to search for active FFL dealers by zip code, display dealer information, and verify license status. Selected FFL is stored on the order and included in the disposition record.

State Restriction Engine: A configurable rules table that maps products (or product categories) to state-level restrictions. During checkout, the engine checks the customer's shipping state against applicable rules and blocks or warns as appropriate. Rules are maintained in the Medusa admin and can be updated without code changes.

Cost: FastBound pricing ranges from approximately $39/month to $99/month depending on the plan and volume. FFLScope/FFL lookup services have separate pricing, typically under $50/month for moderate volume.

9. Email Marketing (Klaviyo)

Klaviyo is the email and SMS marketing platform for the store. It handles newsletter management, automated marketing flows, customer segmentation, and campaign analytics. A Medusa.js plugin exists for event-driven data syncing.

9.1 Integration Architecture

The Klaviyo-Medusa plugin listens to Medusa events (order placed, order fulfilled, cart abandoned, customer created, product updated) and pushes structured data to Klaviyo's API. This enables automated marketing flows triggered by real customer behavior.

9.2 Automated Flows

Flow	Description & Trigger
Welcome Series	Triggered on newsletter signup or account creation. 3-5 emails introducing the brand, featured products, manufacturer highlights, and a CTA to browse.
Abandoned Cart	Triggered when a customer adds items to cart but does not complete checkout within a configurable window (e.g., 4 hours). Includes product images, name, price from the cart.
Post-Purchase	Triggered on order fulfillment. Includes order summary, tracking information, care/maintenance tips for the specific product type, and a prompt to leave feedback.
Back-in-Stock	Triggered when a previously out-of-stock product is relisted. Sent to customers who viewed or inquired about the item.
Browse Abandonment	Triggered when a customer views a product page multiple times without adding to cart. Sends a gentle reminder with the product details.
VIP/Collector Segment	Triggered by purchase history thresholds. High-value customers receive early access to new arrivals, exclusive manufacturer stories, and priority inquiry handling.
New Arrival Announcement	Triggered when new products are published. Segmented by brand/category preferences derived from browse and purchase history.

9.3 Newsletter Management

Newsletter signup forms are embedded on the Next.js storefront (footer, popup, dedicated page) and submit directly to Klaviyo's API. Subscribers are tagged by source (homepage, product page, blog post) for segmentation. Newsletter campaigns are composed in Klaviyo's email builder using brand templates that match the storefront design. Blog posts from Payload CMS can feed into newsletter content via Klaviyo's content feeds feature.

Cost: Klaviyo is free for up to 250 contacts. Paid plans start at $20/month for 500 contacts and scale with list size. At 5,000 contacts, expect approximately $100/month. At 25,000 contacts, approximately $400/month.

10. Transactional Email (Resend + React Email)

Transactional emails (order confirmations, shipping notifications, product inquiry acknowledgments, password resets) are handled separately from marketing email via Resend, paired with React Email for template development.

Why separate from Klaviyo: Transactional emails require guaranteed immediate delivery (not batch-queued) and are not subject to unsubscribe rules. Resend is built for this: fast delivery, high deliverability, and developer-friendly templates.

React Email: Email templates are built in React (the same technology as the front-end), ensuring visual consistency with the storefront. Templates for: order confirmation (with line items, prices, FFL details), shipping notification (with tracking number and carrier link), payment received confirmation (for wire/check), product inquiry received (with product details for both customer and internal team), and account creation.

Product Inquiry Email: When a customer submits an inquiry form on a product page, the internal notification email includes the full product record: product name, SKU, images, current price, stock status, manufacturer, caliber, and the customer's message. This eliminates the need for the team to look up the product separately.

Cost: Resend starts free (100 emails/day). Paid plans begin at $20/month for 50,000 emails/month, scaling as needed.

11. Video Platform (Mux)

Mux is an API-first video infrastructure platform. It handles video upload, encoding, adaptive bitrate streaming, thumbnail generation, and analytics. Unlike YouTube/Vimeo embeds, Mux provides a brandless, high-performance player that keeps visitors on your site.

11.1 Product Video Implementation

Upload & Encoding: Videos are uploaded via the Mux API (either from the Medusa admin or a custom upload interface). Mux automatically encodes to multiple quality levels for adaptive streaming.

Player (Mux Player): An embeddable, customizable HTML5 player. Styled to match the storefront. No third-party branding, no recommended videos, no ads. Supports poster frames, captions, and keyboard accessibility.

Thumbnail Generation: Mux generates still frames from the video that can be used as poster images on product listing pages, in email campaigns, and in social sharing previews.

Video Types: Product walkthrough videos (30-90 seconds showing all angles, action, detail shots), manufacturer feature videos (brand storytelling), and eventually live stream events (new arrival showcases, collector Q&As).

Cost: Mux pricing is usage-based: $0.007/minute stored, $0.00075/minute streamed. For a catalog of 50 product videos averaging 60 seconds each, with moderate streaming traffic, expect approximately $20-60/month.

12. 3D Product Visualization & Augmented Reality

Interactive 3D product viewing with augmented reality is the single most differentiating feature in the firearms e-commerce space. No competitor offers this capability. The implementation uses a combination of open-source web standards and optional advanced rendering.

12.1 Technology Stack for 3D

Google model-viewer (Primary): An open-source web component maintained by Google. Provides a polished 3D viewer with rotate, zoom, and pan controls. Includes built-in AR activation: on iOS, it launches AR Quick Look with USDZ files; on Android, it launches Scene Viewer with glTF files. No app download required for the customer. Integrates into Next.js product pages as a standard HTML element.

React Three Fiber (Advanced Experiences): A React renderer for Three.js, used for advanced interactive experiences: clickable hotspots on the firearm (tap the trigger for specs, tap the slide for finish details), animated field-strip or disassembly views, exploded-view diagrams, and comparison overlays between models. This is Phase 4 work and optional.

3D Asset Formats: glTF/GLB for web (universal standard, supported by all browsers and AI/search platforms). USDZ for iOS AR (Apple's format, required for AR Quick Look). Both are generated from the same source model.

12.2 3D Asset Creation Pipeline

Photogrammetry (Recommended for Firearms): Capture 50-100 high-resolution photographs of the firearm from all angles using a turntable rig and consistent lighting. Process through photogrammetry software (Polycam Pro, RealityScan by Epic Games, or Agisoft Metashape for professional results) to generate a 3D model. Clean up in Blender (free, open source), optimize for web (retopology, PBR texturing, Draco compression), and export to glTF + USDZ. Target file size: 2-5 MB per model.

Professional 3D Scanning Services: For initial catalog, consider contracting a 3D scanning service (Modelry, CGTrader, or local). Cost: approximately $200-800 per model depending on complexity and quality. Recommended for the first 5-10 flagship products to establish quality standards.

In-House Pipeline (Long-term): Once the process is established, new products can be scanned in-house using an iPhone/iPad Pro (LiDAR scanner) with Polycam Pro ($50/month) and refined in Blender. This reduces per-model cost to near-zero after initial learning investment.

12.3 Integration with Product Pages

The product page conditionally renders the 3D viewer: if a glTF model exists for the product, the model-viewer component appears in the product media gallery alongside photographs and video. The viewer includes AR activation buttons for mobile devices. Metadata on the product record tracks which products have 3D models, their file URLs, and display settings. Products without 3D models display the standard photo gallery and video. This allows incremental rollout: launch with photography, add 3D to select products over time.

13. SEO, Schema & Structured Data

Search engine optimization and structured data are handled at both the framework level (Next.js) and the content level (Payload CMS and Medusa.js product data).

13.1 Technical SEO (Next.js)

Server-side rendering ensures all content is visible to crawlers without JavaScript execution. Dynamic meta tags (title, description, OpenGraph, Twitter Card) are generated per-page from CMS and commerce data. Automatic XML sitemap generation includes all products, categories, articles, and manufacturer pages with last-modified dates. Robots.txt is configured to allow all major search engine and AI crawlers. Canonical URLs prevent duplicate content issues. Structured breadcrumbs are generated for all product and category pages.

13.2 Schema Markup (JSON-LD)

Every page type includes appropriate Schema.org markup injected as JSON-LD in the page head. This is auto-generated from the product/content data, not manually maintained.

Page Type	Schema Types & Key Properties
Product Detail	Product schema: name, description, image, brand, manufacturer, sku, mpn, offers (price, availability, condition, priceCurrency), aggregateRating, review. Custom properties: caliber, barrelLength, actionType, finish
Category/Collection	CollectionPage schema: name, description, numberOfItems, mainEntity (list of Product references)
Blog/Article	Article schema: headline, author, datePublished, dateModified, image, publisher, description, mainEntityOfPage
Manufacturer Profile	Organization schema: name, description, logo, foundingDate, foundingLocation, url, sameAs (social links)
FAQ/Resource	FAQPage schema: mainEntity (array of Question + acceptedAnswer pairs)
Site-Wide	Organization, WebSite, SearchAction schemas on all pages. BreadcrumbList on all interior pages

14. LLM Integration: llms.txt & MCP Server

This section covers the AI-forward capabilities that position the platform for agentic commerce and AI-driven product discovery.

14.1 llms.txt Auto-Generation

A scheduled job (daily or on product publish) generates the /llms.txt and /llms-full.txt files from the Medusa product database and Payload CMS content. The file follows the emerging llms.txt specification: markdown format with an H1 identifying the site, a brief description, and H2 sections organizing links by type (Products, Categories, Brands, Articles, Policies). Each link includes a description helping AI models understand the content. The extended version (/llms-full.txt) includes complete product details (specifications, pricing, availability) for models that want deeper context.

14.2 MCP Server (Model Context Protocol)

The MCP server is a custom TypeScript service that exposes the product catalog and store information to AI agents via the MCP standard. This is the foundational infrastructure for agentic commerce.

Capabilities Exposed: Product search (by brand, category, caliber, price range, availability), product detail retrieval (full specs, pricing, stock status, images), brand/manufacturer information, store policies (shipping, returns, FFL requirements), and availability checking.

Security: The MCP server exposes read-only catalog data. No customer PII, no order data, and no admin actions are accessible. Rate limiting prevents abuse. Tool-level scoping controls what external agents can query.

Use Case: When a potential buyer asks ChatGPT, Claude, or Perplexity "where can I buy a Nighthawk Custom Treasurer in .45 ACP," the AI agent can query the MCP server, receive structured product data, confirm availability, and direct the buyer to the product page with accurate pricing and stock information.

Future Expansion: As agentic checkout protocols mature (UCP, ACP), the MCP server can be extended to support cart creation and checkout initiation by AI agents. When firearms-friendly payment processors adopt these standards, the platform is ready to participate.

14.3 AI Crawler Configuration

The robots.txt file explicitly allows major AI crawlers (GPTBot, ClaudeBot, PerplexityBot, Google-Extended, ChatGPT-User, Applebot-Extended) access to product pages, category pages, blog content, and manufacturer profiles. Admin, account, and checkout pages are disallowed. The llms.txt file is referenced in robots.txt. Server logs are monitored for AI crawler activity to track adoption and adjust strategy.

15. Newsletter & Blog System

The blog and newsletter operate as interconnected systems: Payload CMS handles content creation and management; the Next.js front-end renders the blog; and Klaviyo manages subscriber lists and email delivery.

15.1 Blog Architecture

Blog posts are created in the Payload CMS admin with a rich text editor supporting formatted text, images, embedded video (Mux), product embeds (pull live product card from Medusa), and custom content blocks. Each post has SEO fields, category/tag taxonomy, author attribution, and publish scheduling. The Next.js front-end renders the blog index (paginated, filterable by category/tag), individual post pages, and an RSS feed. Posts automatically apply internal linking rules from the linking engine.

15.2 Newsletter Integration

Signup forms are embedded in the storefront footer (persistent), a timed popup (configurable delay, dismissable, cookie-respecting), dedicated signup page, and inline CTAs within blog posts. All forms submit to Klaviyo's API with source tagging. Subscribers are double opt-in for compliance. Blog content feeds into Klaviyo campaigns via content feeds or manual curation in the Klaviyo editor.

16. Product Inquiry Forms

Every product page includes an inquiry form that allows potential buyers to ask questions about a specific item. The form is pre-populated with the product context so the customer doesn't need to re-type what they're asking about.

Form Fields: Name, email, phone (optional), message/question. Hidden fields: product ID, product name, SKU, current price, stock status, product URL.

Customer Experience: Customer fills out the form on the product page. Receives an immediate confirmation email acknowledging their inquiry and referencing the specific product.

Internal Notification: The Luxus team receives an email via Resend containing: the customer's contact information, their message, and the full product record (name, images, SKU, price, stock status, manufacturer, caliber, any special notes). This email is actionable without needing to look anything up in the admin.

CRM Logging: Inquiries are logged in Medusa as customer interactions and synced to Klaviyo. If the customer later signs up or makes a purchase, their inquiry history is associated with their profile.

17. Internal Site Linking Engine

The internal linking engine is a custom Payload CMS plugin that automatically inserts contextual hyperlinks into blog posts and editorial content based on a configurable rules table.

Rules Table: Maintained in Payload CMS admin. Each rule has: a keyword or phrase (e.g., "Nighthawk Custom"), a target URL (e.g., /brands/nighthawk-custom), a maximum link count per page (e.g., 1), and a priority (for conflicting keywords). Rules can target product pages, category pages, brand pages, or articles.

Processing: When a blog post is saved/published, the engine scans the rich text body, identifies matching keywords (case-insensitive, whole-word), and inserts links according to the rules. It respects the max-per-page limit and does not link within headings, existing links, or image captions.

SEO Impact: Internal linking distributes page authority across the site, helps search engines understand topical relationships, and improves crawl depth. When combined with the blog content strategy, this creates a strong organic search foundation.

18. Order Tracking & Invoicing

Order Lifecycle: Order Created (cart submitted) → Payment Pending (wire/check) or Payment Authorized (credit card) → Payment Captured → Fulfillment (item shipped to receiving FFL) → Delivered → Completed. Each state transition triggers a customer notification email via Resend.

Tracking Integration: When the team ships an order, the tracking number and carrier (FedEx/UPS, both support firearms with proper labeling) are entered in the Medusa admin. The customer receives a shipping notification email with tracking link. The account dashboard shows order status with tracking.

PDF Invoicing: A custom Medusa module generates branded PDF invoices using a library like pdf-lib or Puppeteer. Invoices include: Luxus sub-brand logo, order number, date, customer details, receiving FFL details, line items with serial numbers, payment method and amount, tax breakdown, and FFL license information. Invoices are attached to order confirmation emails and available for download in the customer account.

19. Infrastructure & Hosting

The platform uses a distributed hosting architecture optimized for performance, reliability, and cost efficiency.

Component	Service	Plan/Tier	Est. Monthly Cost
Front-End (Next.js)	Vercel	Pro ($20/mo) or Team ($150/mo)	$20 - $150
Backend (Medusa + Payload + MCP)	Railway or Render	Pro tier with autoscaling	$40 - $100
Database (PostgreSQL)	Neon or Supabase	Pro tier	$10 - $25
File Storage (Images, 3D, Video)	Cloudflare R2 or AWS S3	Usage-based	$5 - $20
CDN & Security	Cloudflare	Pro ($20/mo) or Free tier	$0 - $20
Search Engine (Meilisearch)	Meilisearch Cloud or self-hosted on Railway	Build plan	$0 - $30
Error Monitoring	Sentry	Team plan	$0 - $26
Domain & DNS	Cloudflare Registrar	Registration	~$1
SSL/TLS	Cloudflare / Vercel auto-SSL	Included	$0

Estimated Total Hosting: $76 - $372/month depending on traffic volume and tier selections. This is significantly lower than BigCommerce Enterprise ($1,000-1,500/month) or firearms-specific platforms ($149-999/month) while providing superior performance and flexibility.

20. Mobile Experience & Progressive Web App

Mobile devices account for 65-75% of e-commerce traffic. The platform is built mobile-first: every page, component, and interaction is designed for touchscreens first, then adapted upward to tablet and desktop. This is not a separate mobile site or a responsive afterthought; mobile is the primary design target.

20.1 Mobile-First Design Approach

The Next.js + React + Tailwind CSS stack renders a single codebase that adapts to all screen sizes via responsive breakpoints. There is no separate mobile subdomain, no separate codebase, and no app store dependency. The designer delivers mobile wireframes and mockups first for every page type, with desktop layouts derived from the mobile foundation.

Product Detail Page (Mobile): The PDP is the single most important mobile design deliverable. On mobile, the product gallery becomes a full-width edge-to-edge swipeable carousel with pinch-to-zoom. Specifications collapse into an accordion layout. The inquiry form and add-to-cart button use a sticky bottom bar that remains visible during scroll. Tabs for switching between photography, video, and 3D model use a horizontal swipe strip.

Product Listing Page (Mobile): Products display in a single-column or two-column grid (user preference toggle). Filters slide in from the bottom as a drawer overlay rather than occupying page space. Sort options use a compact dropdown. Infinite scroll or progressive loading replaces pagination for seamless browsing.

Navigation (Mobile): The desktop mega-menu is replaced with a full-screen slide-out drawer featuring clear hierarchy, prominent search bar, quick-access category links, and brand shortcuts. Search uses a full-screen overlay with Meilisearch real-time results appearing as the user types, filterable by brand, caliber, and category without page reload.

20.2 Touch-Optimized 3D & AR

The model-viewer 3D component is natively touch-optimized: single-finger drag to rotate, pinch to zoom, two-finger drag to pan. The AR activation button is placed prominently on mobile product pages with clear labeling (e.g., "View in Your Space"). Tapping it launches the phone camera directly into the AR experience with no app download required (iOS uses AR Quick Look with USDZ; Android uses Scene Viewer with glTF).

The AR experience is the signature mobile differentiator. No firearms retailer offers this. A collector can see a Nighthawk Custom Treasurer rendered at true scale on their desk, rotate it, and examine it from every angle through their phone camera. The surrounding UI (media tab switching, AR button placement, loading states) is designed specifically for one-handed mobile operation.

20.3 Mobile Checkout Optimization

Checkout is where mobile conversions are won or lost. The platform implements a progressive single-page checkout flow rather than multi-page navigation, minimizing load times and reducing abandonment.

Autofill & Digital Wallets: All form fields support browser autofill for address, email, and phone. Apple Pay and Google Pay integration (if supported by the chosen firearms processor) allows one-tap payment without manual card entry. Confirm digital wallet support during payment processor evaluation.

GPS-Based FFL Finder: On mobile, the FFL dealer selection step uses the device GPS to automatically show nearby dealers on a map view, sorted by distance. The customer taps to select rather than typing a zip code. Fallback to manual zip entry for users who decline location permission.

Wire Transfer on Mobile: When a customer selects wire transfer, the confirmation screen provides a "Copy Bank Details" button that copies all wire instructions to the clipboard in one tap, plus a "Email Instructions to Myself" option. This is critical for high-ticket items where the customer will complete the wire from a banking app or desktop later.

Minimal Form Fields: Every field on mobile must justify its existence. Optional fields are hidden behind a "Show more" toggle. Phone number input uses the numeric keyboard. State selection uses a native picker rather than a dropdown. The goal is checkout completion in under 90 seconds for returning customers.

20.4 Progressive Web App (PWA)

Instead of building native iOS and Android apps (which cost $80,000-$200,000, require separate codebases, and introduce App Store firearms policy risk), the platform is built as a Progressive Web App. A PWA uses modern web capabilities to deliver an app-like experience from the browser.

PWA Capability	Implementation
Home Screen Installation	Users are prompted to "Add to Home Screen" after their second visit. The store appears as a standalone app with the Luxus sub-brand icon, splash screen, and no browser chrome. Launches instantly on tap.
Push Notifications	Opt-in notifications for: back-in-stock alerts on favorited items, order status updates (payment confirmed, shipped, delivered), new arrival announcements from preferred brands, and price changes on wishlist items. Delivered via the Web Push API through the service worker.
Offline Browsing	Previously viewed product pages, manufacturer profiles, and blog articles are cached via the service worker and remain accessible without an internet connection. Product images are cached progressively. Cart contents persist locally.
Fast Re-Engagement	Tapping the home screen icon loads the store instantly from the service worker cache, with fresh data fetched in the background. No cold browser startup or URL typing.
App-Like Transitions	Page transitions use smooth animations rather than full page loads. Pull-to-refresh on listing pages. Native-feeling scroll behavior and momentum.

Technical Implementation: Next.js supports PWA via the next-pwa package or Serwist. Configuration includes: a Web App Manifest (manifest.json) defining the app name, icons, theme color, and display mode; a Service Worker for caching strategies (network-first for product data, cache-first for images and static assets); and a push notification backend that connects to Medusa order events and Klaviyo for marketing notifications.

20.5 Performance Budgets

Hard performance targets are set for mobile devices and enforced during development via Lighthouse CI in the deployment pipeline. These targets are based on Google Core Web Vitals thresholds that directly impact both search rankings and user experience.

Metric	Target	Why It Matters
Largest Contentful Paint (LCP)	Under 2.5 seconds	Time until the main product image or hero is visible. Slow LCP causes immediate bounce on mobile.
First Input Delay (FID)	Under 100ms	Time until the page responds to the first tap. Laggy interaction feels broken on touch devices.
Cumulative Layout Shift (CLS)	Under 0.1	Prevents content from jumping around as images and ads load. Layout shifts cause mis-taps on mobile.
Time to First Byte (TTFB)	Under 200ms	Server response time. Edge deployment on Vercel ensures this globally. Also critical for AI crawler performance.
Total Page Weight (PDP)	Under 1.5 MB initial load	3D models, videos, and below-fold images lazy-load after the initial paint. Only critical assets load upfront.
Speed Index	Under 3.0 seconds	Visual completeness of the page. Measures perceived load speed, not just technical metrics.

Testing Protocol: Performance is tested on real mid-range Android devices (e.g., Samsung Galaxy A54 or equivalent) over throttled 4G connections, not just desktop Chrome DevTools. Lighthouse CI runs automatically on every deployment via the Vercel/GitHub integration. Any deployment that regresses below the performance budget triggers a build warning.

20.6 Mobile-Specific UX Patterns

Sticky Add-to-Cart Bar: On product pages, a compact bar with the price and "Add to Cart" button sticks to the bottom of the screen as the user scrolls through specifications, description, and reviews. Always accessible, never buried.

Swipe Gestures: Product gallery images are navigated by swiping left/right. The media type tabs (Photo, Video, 3D) also respond to horizontal swipe. Back navigation respects the native swipe-from-edge gesture on both iOS and Android.

Bottom Sheet Modals: On mobile, filters, sort options, FFL selection, and the inquiry form open as bottom sheets that slide up from the screen edge, following native mobile interaction patterns rather than traditional desktop modal overlays.

Tap Targets: All interactive elements (buttons, links, form fields) meet the minimum 48x48px tap target size specified by WCAG accessibility guidelines. Spacing between adjacent tap targets prevents accidental taps.

Image Zoom: Product photography supports pinch-to-zoom that opens a full-screen lightbox with pan capability. Essential for inspecting finish details, engravings, and serial numbers on high-end firearms.

20.7 Why Not a Native App

A native iOS/Android app is not recommended for the initial build for several critical reasons. Cost: native apps for both platforms would add $80,000-$200,000 to the build and require ongoing maintenance of three separate codebases (web, iOS, Android). App Store policies: both Apple and Google have firearms-related content policies that could restrict or reject the app, reintroducing the same platform risk the composable web stack eliminates. Audience size: at launch volume, the install friction of downloading an app from a store is a conversion barrier that a PWA avoids entirely. Feature parity: PWAs now support push notifications, offline access, home screen installation, camera access (for AR), and payment APIs, covering all capabilities a native app would provide for this use case.

If mobile engagement metrics (monthly active users, repeat visit frequency, push notification opt-in rates) justify it in the future, a native app can be evaluated as a Phase 5+ initiative using React Native, which shares code with the existing React/Next.js front-end to minimize additional development cost.

21. Cross-Site Integration with luxuscap.com

The sub-brand store and luxuscap.com operate as sibling properties with strategic cross-linking. The gallery site remains the curated collectible reference; the store is the transactional arm for production high-end firearms.

Gallery-to-Store Linking: luxuscap.com includes a navigation link to the store (e.g., "Shop Available Firearms" in the header/menu). Individual gallery items that overlap with store inventory can include an "Available for Purchase" badge linking to the store product page.

Store-to-Gallery Linking: The store references luxuscap.com for brand credibility: "From the curators of Luxus Capital" messaging, links to the gallery for brands/items that exist in the collection but are not for sale, and shared article content where relevant.

SEO Considerations: The two sites target different search intents (research/gallery vs. buy/purchase). Cross-domain linking passes authority between properties. Canonical URLs ensure no duplicate content issues for any shared descriptions or images.

Technical Integration: Since luxuscap.com is WordPress-based, a lightweight REST API bridge can sync availability status between the Medusa store and WordPress. When a product is listed on the store, the corresponding gallery item on luxuscap.com can show a real-time "Available" badge. This is optional Phase 3+ work.

22. Development Phases & Timeline

The build is structured in phases that deliver incremental value. Phase 1 gets the store live and selling. Each subsequent phase adds capability without disrupting operations.

Phase 1: Core Commerce Launch (Weeks 1-8)

Goal: Live, functional store with products listed, payments accepted, and orders flowing.

Deliverable	Details
Medusa.js Setup & Configuration	Install, configure, deploy. Product schema with firearms attributes. Admin dashboard customization.
Next.js Storefront	Homepage, PLP, PDP, cart, account pages. Mobile-responsive, server-rendered. Brand design implementation.
Payload CMS Setup	Install, configure. Blog and manufacturer page content types. SEO fields on all types.
Payment Integration	Firearms-friendly processor gateway integration. Wire/check manual payment flow with reservation logic.
FastBound Integration	A&D book API sync. Acquisition and disposition logging.
FFL Dealer Selection at Checkout	FFL lookup integration. Dealer selection step in checkout flow.
State Restriction Engine	Configurable rules table. Checkout enforcement.
Product Inquiry Forms	Per-product form with context injection. Internal email with full product data. Customer confirmation email.
Transactional Email Templates	Resend + React Email templates for: order confirmation, shipping notification, payment received, inquiry received, account creation.
Basic SEO Setup	Meta tags, sitemap, robots.txt, JSON-LD Product schema, breadcrumbs.
Mobile-First Design Implementation	Mobile wireframes and mockups delivered first for all page types. Responsive layouts with Tailwind CSS breakpoints. Touch-optimized tap targets (48x48px minimum). Swipeable product gallery. Sticky add-to-cart bar on PDP.
Mobile Checkout Optimization	Single-page progressive checkout flow. Browser autofill support. GPS-based FFL dealer finder. Wire transfer copy-to-clipboard instructions. Apple Pay / Google Pay integration (if processor supports).
Progressive Web App (PWA) Setup	Web App Manifest configuration (icons, theme color, display mode). Service worker for caching strategy. Home screen install prompt. Offline support for cached pages.
Performance Budget Enforcement	Lighthouse CI integration in deployment pipeline. Core Web Vitals targets: LCP under 2.5s, FID under 100ms,

	CLS under 0.1. Testing on real mid-range Android devices over 4G.
Initial Product Data Entry	Load initial product catalog (images, descriptions, specs, pricing).
Deployment & QA	Production deployment on Vercel + Railway. Cross-browser testing. Mobile testing. Payment flow testing.

Phase 2: Content & Marketing Engine (Weeks 9-14)

Goal: Blog publishing, email marketing automation, newsletter growth, and internal linking driving organic traffic.

Deliverable	Details
Klaviyo Integration	Medusa plugin setup. Event sync (orders, carts, customers). Subscriber list configuration.
Automated Email Flows	Welcome series, abandoned cart, post-purchase, back-in-stock, browse abandonment. Template design matching storefront.
Newsletter System	Signup forms (footer, popup, inline). Double opt-in flow. Source tagging. Campaign template.
Blog Launch	Payload CMS blog content type finalized. Blog index and post pages on Next.js. RSS feed. Category/tag pages.
Manufacturer Profile Pages	Content type in Payload. Rendered pages on Next.js with linked products from Medusa.
Internal Linking Engine	Payload plugin: rules table, keyword matching, auto-insertion on publish. Initial rules configuration.
Cross-Site Links with luxuscap.com	Navigation links between sites. Shared branding elements. Initial cross-linking strategy.
Analytics Setup	Plausible or Fathom installation. Event tracking for add-to-cart, checkout start, purchase, inquiry submit.
PWA Push Notifications	Web Push API integration via service worker. Opt-in prompts. Notification types: back-in-stock, order updates, new arrivals. Connected to Medusa events and Klaviyo segments.

Phase 3: AI Integration Layer (Weeks 15-20)

Goal: Platform is discoverable and queryable by AI agents. Structured data is comprehensive and machine-optimized.

Deliverable	Details
llms.txt Generator	Automated job that generates /llms.txt and /llms-full.txt from Medusa product data and Payload content. Updates on product publish or daily.

MCP Server	Custom TypeScript service implementing MCP spec. Exposes: product search, product detail, brand info, availability check. Rate limiting. Read-only access.
Enhanced Schema Markup	Extend JSON-LD on all page types. Add FAQ schema to resource pages. Verify with Google Rich Results Test.
AI Crawler Optimization	robots.txt fine-tuning for AI bots. Server log monitoring for GPTBot, ClaudeBot, PerplexityBot activity. TTFB optimization.
Meilisearch Implementation	Full-text product search with faceted filtering (brand, caliber, category, price range). Instant search UI on storefront.
luxuscap.com Availability Bridge	REST API bridge syncing product availability status between Medusa and WordPress for real-time badges on gallery items.

Phase 4: Rich Media & Differentiation (Weeks 21-28)

Goal: Industry-leading product presentation with video, 3D, and AR capabilities.

Deliverable	Details
Mux Video Integration	Video upload pipeline. Mux Player embedded on PDP. Poster frame generation. Video on 10-20 initial products.
3D Product Viewer (model-viewer)	model-viewer component on PDP. Conditional rendering (only for products with 3D assets). AR activation for iOS and Android.
3D Asset Pipeline Setup	Photogrammetry workflow established (equipment, software, process). First 5-10 products scanned and optimized.
React Three Fiber Advanced Viewer (Optional)	Interactive hotspots, exploded views, or animated disassembly for select flagship products.
Video Content Production Plan	Equipment list, shooting guide, and template for consistent product video production by the Luxus team.

Phase 5: Ongoing Optimization & Expansion

Goal: Continuous improvement, new capabilities, and agentic commerce readiness.

Deliverable	Details
On-Site AI Assistant (Conversational Commerce)	Chat widget powered by Claude or similar LLM, querying the MCP server for real-time product answers. Collector Q&A, product recommendations.
Agentic Checkout Protocols	As firearms-friendly processors adopt UCP/ACP standards, extend the MCP server to support agent-initiated checkout.
Customer Loyalty/VIP Program	Tiered system for repeat collectors. Early access, exclusive content, priority inquiries.

Multi-Language Support	If expanding internationally. Next.js i18n routing, Medusa multi-currency, Payload localized content.
A/B Testing & CRO	Conversion rate optimization on PDP, checkout, and inquiry forms using Vercel's experimentation features.
Collector Companion PWA Experience	Enhanced mobile experience for registered collectors: personalized push notifications for items matching interests, personal vault of favorited items, priority access to new arrivals, and collector profile with purchase history. Evaluates native app (React Native) if engagement metrics justify the investment.
Expanded 3D Catalog	Scale photogrammetry pipeline to cover full catalog. Continuous improvement of model quality.

23. Cost Estimates: Build & Monthly Operations

23.1 Development Build Costs (One-Time)

These estimates assume a senior full-stack developer (or small agency) using AI-assisted development tools, which significantly accelerate front-end and integration work.

Phase	Estimated Hours	Rate Range	Cost Range
Phase 1: Core Commerce + Mobile	240-360 hours	$100-175/hr	$24,000 - $63,000
Phase 2: Content & Marketing	100-160 hours	$100-175/hr	$10,000 - $28,000
Phase 3: AI Integration	80-120 hours	$100-175/hr	$8,000 - $21,000
Phase 4: Rich Media	80-140 hours	$100-175/hr	$8,000 - $24,500
Total Build Estimate	**500-780 hours**		**$50,000 - $136,500**

Note: The wide range reflects the difference between a solo senior developer with AI tooling (lower end) and a multi-person agency with project management overhead (higher end). AI-assisted development (Claude Code, Cursor, Copilot) reduces implementation time by an estimated 30-50% compared to traditional development for this type of project.

23.2 Monthly Operating Costs

Service	Tier	Monthly Cost	Annual Cost
Hosting (Vercel + Railway + DB)	Pro tiers	$70 - $275	$840 - $3,300
Cloudflare (CDN + DNS + Security)	Pro or Free	$0 - $20	$0 - $240
File Storage (R2/S3)	Usage-based	$5 - $20	$60 - $240
Payment Processor	Firearms-friendly	$20 + transaction fees	$240+
FastBound (Compliance)	FFL plan	$39 - $99	$468 - $1,188
Klaviyo (Email Marketing)	Based on contacts	$20 - $400	$240 - $4,800
Resend (Transactional Email)	Pro plan	$20 - $50	$240 - $600
Mux (Video)	Usage-based	$20 - $60	$240 - $720
Meilisearch Cloud	Build plan	$0 - $30	$0 - $360
Analytics (Plausible/Fathom)	Growth plan	$9 - $14	$108 - $168
Sentry (Monitoring)	Team plan	$0 - $26	$0 - $312
GitHub	Team plan	$0 - $16	$0 - $192
3D Tools (Polycam Pro)	Monthly sub	$0 - $50	$0 - $600
Total Monthly Operating		**$203 - $1,060**	**$2,436 - $12,720**

Note: Transaction fees from the payment processor are additional and vary by processor and volume. Typical firearms processing rates range from 2.5% to 3.5% per credit card transaction. Wire transfers typically have no processing fee beyond standard bank charges.

23.3 Cost Comparison vs. Alternatives

Platform	Build Cost	Monthly Cost	3-Year Total
Composable (Recommended)	$50,000 - $136,500	$203 - $1,060	$57,308 - $174,660
BigCommerce Enterprise	$15,000 - $40,000 (theme + setup)	$1,000 - $1,800	$51,000 - $104,800
Gearfire Pro	$1,000 (setup fee)	$249/month	$9,964
AmmoReady	$500 - $2,000 (setup)	$179 - $999/month	$6,944 - $37,964

The composable stack has a higher upfront investment but provides: zero platform fees, full data ownership, no firearms policy risk, superior design flexibility, AI-readiness that no alternative offers, and long-term cost predictability. The firearms-specific platforms (Gearfire, AmmoReady) are significantly cheaper but offer limited design control, limited SEO capabilities, no AI integration path, and are optimized for distributor-driven retail rather than premium branded commerce.

24. Team Requirements

24.1 During Build (Phases 1-4)

Senior Full-Stack Developer (1): Proficient in TypeScript, Node.js, React, Next.js. Experience with headless commerce (Medusa preferred but Shopify/Saleor transferable). Comfortable with API integrations and database design. This is the primary builder. With AI-assisted development tools, one strong developer can handle the entire build.

Designer (1, part-time or contract): UI/UX design for the storefront with a mobile-first workflow. Figma deliverables: mobile wireframes first for all page types, then tablet and desktop adaptations. Key screens: PDP (product detail with gallery, 3D, video, sticky cart bar), mobile checkout flow with FFL finder, homepage, PLP with filter drawer. Must understand premium/luxury design language consistent with the Luxus brand and touch-optimized interaction patterns (bottom sheets, swipe gestures, tap targets).

Content/Product Manager (Luxus team): Responsible for product photography, descriptions, pricing, and blog content. Works with the developer on content structure and with the designer on visual standards.

24.2 Ongoing Operations

Developer (part-time, 10-20 hrs/month): Ongoing maintenance, bug fixes, feature enhancements, dependency updates, monitoring. Can be a retainer arrangement with the build developer or a dedicated part-time hire.

Content Manager (Luxus team): Publishes blog posts, manages product listings, handles inquiries, runs Klaviyo campaigns. All via admin dashboards, no code required.

25. Appendix: Complete Software & Service Reference

The following table provides URLs and documentation references for every technology and service specified in this document.

Technology	URL	Documentation
Medusa.js	https://medusajs.com	https://docs.medusajs.com
Next.js	https://nextjs.org	https://nextjs.org/docs
Payload CMS	https://payloadcms.com	https://payloadcms.com/docs
Klaviyo	https://www.klaviyo.com	https://developers.klaviyo.com
Resend	https://resend.com	https://resend.com/docs
React Email	https://react.email	https://react.email/docs
Mux	https://www.mux.com	https://docs.mux.com
Google model-viewer	https://modelviewer.dev	https://modelviewer.dev/docs
React Three Fiber	https://r3f.docs.pmnd.rs	https://r3f.docs.pmnd.rs/getting-started
Three.js	https://threejs.org	https://threejs.org/docs
Meilisearch	https://www.meilisearch.com	https://docs.meilisearch.com
FastBound	https://www.fastbound.com	https://cloud.fastbound.com/docs
Polycam (Photogrammetry)	https://poly.cam	https://poly.cam/docs
Blender (3D Editing)	https://www.blender.org	https://docs.blender.org
Vercel	https://vercel.com	https://vercel.com/docs
Railway	https://railway.app	https://docs.railway.app
Neon (PostgreSQL)	https://neon.tech	https://neon.tech/docs
Supabase (PostgreSQL)	https://supabase.com	https://supabase.com/docs
Cloudflare R2	https://www.cloudflare.com/r2	https://developers.cloudflare.com/r2
Cloudflare	https://www.cloudflare.com	https://developers.cloudflare.com
Sentry	https://sentry.io	https://docs.sentry.io
Plausible Analytics	https://plausible.io	https://plausible.io/docs
GitHub	https://github.com	https://docs.github.com
EPIC Merchant Systems	https://www.epicmerchantsystems.com	Contact for API docs
TacticalPay	https://www.tacticalpay.com	Contact for API docs

PaymentCloud	https://paymentcloudinc.com	Contact for API docs
MCP Specification	https://modelcontextprotocol.io	https://modelcontextprotocol.io/docs
llms.txt Specification	https://llmstxt.org	https://llmstxt.org
next-seo	https://github.com/garmeeh/next-seo	README on GitHub
pdf-lib (Invoices)	https://pdf-lib.js.org	https://pdf-lib.js.org
Tailwind CSS	https://tailwindcss.com	https://tailwindcss.com/docs
next-pwa	https://github.com/shadowwalker/next-pwa	README on GitHub
Serwist (PWA)	https://serwist.pages.dev	https://serwist.pages.dev/docs
Web Push API	https://developer.mozilla.org/en-US/docs/Web/API/Push_API	MDN Web Docs
Lighthouse CI	https://github.com/GoogleChrome/lighthouse-ci	README on GitHub
Web App Manifest	https://developer.mozilla.org/en-US/docs/Web/Manifest	MDN Web Docs

End of Document

Luxus Capital, LLC — Confidential